

UNIVERSITAS CAKRAWALA

LAPORAN INDIVIDU

UJIAN TENGAH SEMESTER (UTS)

Struktur Data & Data Resep

Cookbook System - Aplikasi Manajemen Resep Berbasis Python

Nama	Alexander Tristan
NIM	24110300063
Mata Kuliah	Programming Fundamental
Program Studi	Ilmu Komputer
Dosen Pengampu	Fikri Hanif, S.T., M.S.M.
Semester / Kelas	Ganjil / Prog4
Tahun Akademik	2026/2027

Oktober 2026

1. Kontribusi dan Analisis IPO

Saya mengerjakan fondasi data pada berkas data.py, tepatnya list cook_book berisi 14 resep awal, bentuk dictionary satu resep, pembuat ID acak generate_id(), dan penghubung nomor urut ke resep get_recipe_by_number(). Dua fungsi milik Ikmal dan struktur trie juga tinggal di berkas yang sama, tetapi dua fungsi dan seluruh bentuk data resep itu tanggung jawab saya.

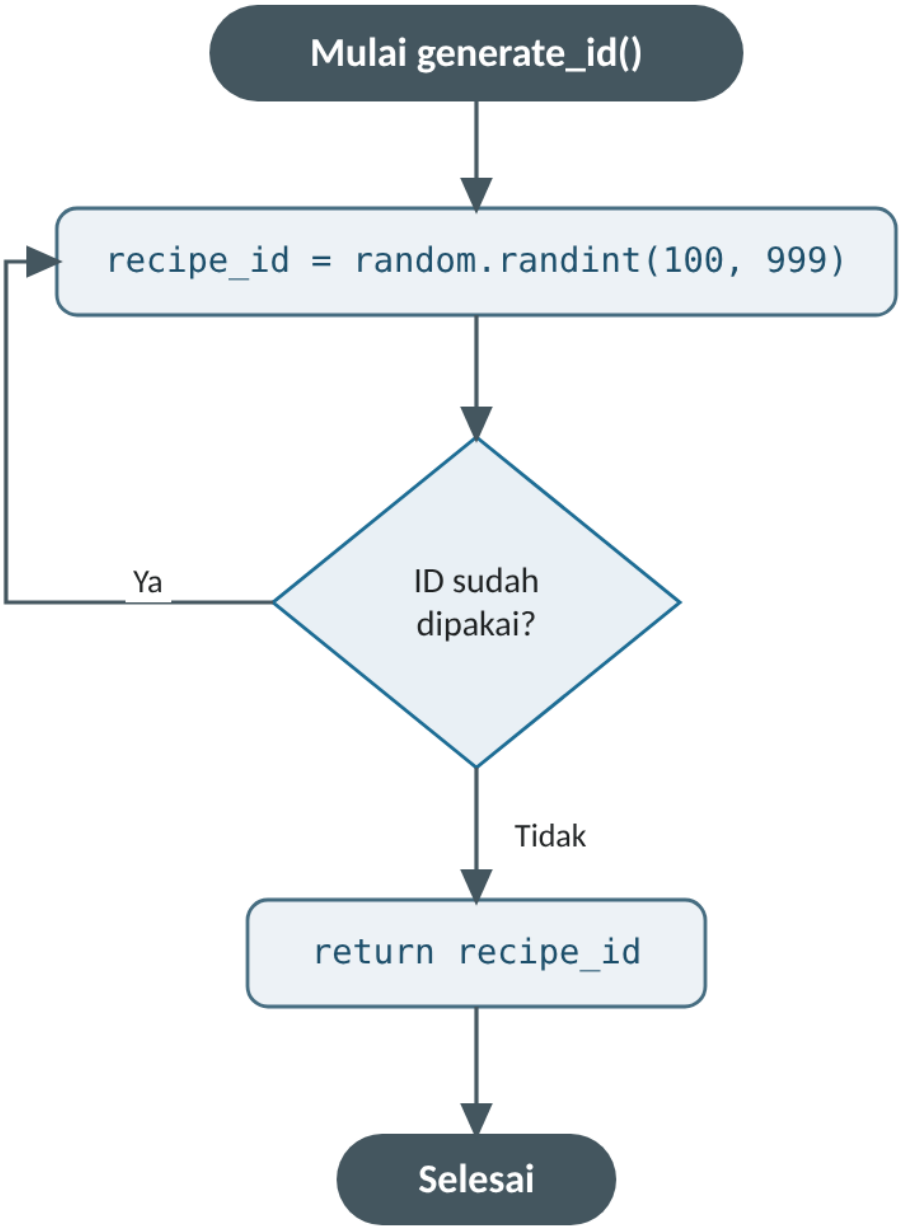
Masalah yang bagian ini selesaikan adalah kebutuhan satu sumber data tunggal yang bentuknya konsisten. Tanpa format yang seragam, setiap anggota akan menyimpan resep dengan cara masing-masing dan penggabungan kode pasti kacau. Selain itu, pengguna tidak boleh dipaksa menghafal ID tiga digit, jadi saya menyiapkan nomor urut 1 sampai 14 yang tampil di layar, sementara ID yang sebenarnya dipakai di balik layar oleh hash table dan trie.

Bagian	Rincian
Input	Tidak langsung dari pengguna. Data awal berupa 14 dictionary resep yang saya susun di cook_book, nomor resep (int) masuk lewat fitur hapus dan ubah milik Lars, dan ID baru (int) lahir dari random.randint(100, 999).
Process	get_recipe_by_number() memeriksa batas nomor lalu mengambil resep dengan indeks list. generate_id() mengacak ID lalu mengecek ke hash table milik Ikmal sampai didapat ID yang belum terpakai.
Output	Dictionary resep yang utuh untuk dipakai fitur lain, atau None sebagai tanda tidak ditemukan. ID acak baru diteruskan ke add_recipe() untuk disimpan.

Tabel 1. Analisis IPO struktur data dan data resep

2. Flowchart Fitur

Alur yang paling layak digambarkan dari bagian saya adalah generate_id(), karena di sanalah keputusan diambil berulang kali. Fungsi ini mengacak angka 100 sampai 999, lalu menanyakan ke get_recipe() apakah angka itu sudah menjadi ID suatu resep. Kalau sudah, angka dibuang dan diacak lagi; kalau belum, angka langsung dikembalikan sebagai ID resmi resep baru. Perulangannya tidak punya hitungan pasti, berapa kali diulang tergantung keberuntungan angka acaknya, dan itu sebabnya bentuknya while.



Gambar 1. Flowchart fungsi generate_id() pada data.py

3. Elemen Pemrograman Dasar

Seluruh data resep saya susun sebagai list berisi dictionary, komposisi yang cocok untuk menyimpan data berlabel seperti resep. Rincian variabel utamanya ada di Tabel 2. Yang perlu saya perhatikan adalah campuran tipe di dalam satu struktur: satu resep memuat int untuk ID, str untuk judul dan metode, serta list untuk bahan, dan semuanya diakses lewat kunci dictionary.

Variabel	Tipe	Isi / Contoh
cook_book	list of dict	14 resep awal, bertambah lewat append()
recipe (satu item)	dict	{'ID': 347, 'title': 'Boiled egg', 'ingredients': ['egg', 'water'], 'cooking_method': 'Boiled'}
recipe_number	int	nomor urut tampil di layar, 1 sampai 14
recipe_id	int	ID acak, selalu di antara 100 dan 999
index	int	hasil hash_function(recipe_id), bernilai 0 sampai 9

Tabel 2. Variabel pada struktur data resep

Keluaran bagian saya berupa dictionary resep yang dikembalikan ke fitur lain untuk ditampilkan, sementara urusan cetak ke layar saya serahkan ke bagian milik Dhafi. Masukannya pun datang dari dalam program. Operator yang saya pakai jelas dan sedikit: perbandingan lebih kecil dan lebih besar dengan penghubung or di pemeriksaan batas nomor, pengurangan 1 pada indeks list, dan operator modulo yang berjalan di hash_function() milik Ikmal setiap kali ID perlu dicari. Contoh nyatanya 347 % 10 = 7, sehingga ID 347 selalu ditempatkan di slot 7.

Kutipan file data.py: operator perbandingan dan logika or pada pemeriksaan batas

```
def get_recipe_by_number(recipe_number):
    if recipe_number < 1 or recipe_number > len(cook_book):
        return None

    return cook_book[recipe_number - 1]
```

4. Percabangan dan Perulangan

Percabangan di bagian saya berupa satu if dengan dua syarat yang digabung or: nomor kurang dari 1 atau lebih dari jumlah resep. Kedua syarat itu harus ada karena nomor 0 dan nomor 15 sama-sama tidak sah meskipun ceritanya berbeda, satu terlalu kecil dan satu terlalu besar. Bila salah satu terpenuhi, fungsi mengembalikan None dan fitur pemanggilnya yang mencetak pesan Recipe not found. Struktur if singkat seperti ini cukup, karena hanya ada dua kemungkinan hasil: nomor sah atau tidak sah.

Perulangan while saya pakai di generate_id() dengan pola while True yang keluar lewat return. Setiap putaran mengacak angka baru, dan pengacakan berhenti tepat saat angka yang belum terpakai ditemukan.

Perulangan for banyak dipakai di tingkat aplikasi: make_hash_table() dan make_trie() membaca seluruh isi cook_book satu per satu saat menyusun struktur, dan opsi Show All Recipes memakai for untuk mencetak 14 resep berurutan. Jadi for muncul saat jumlah pengulangannya sudah diketahui dari panjang list, sementara while saat syarat berhentinya ditentukan kondisi.

5. Fungsi dan Struktur Data

Fungsi yang saya bahas adalah get_recipe_by_number(recipe_number). Parameternya satu, bertipe int, yaitu nomor urut resep sebagaimana tampil di layar. Nilai baliknya dua macam: dictionary resep utuh bila nomornya sah, atau None bila tidak. Pemakaian None sebagai penanda kosong ini penting karena fitur hapus dan ubah milik Lars membandingkan hasilnya dengan None untuk memutuskan mencetak pesan gagal atau melanjutkan proses. Jadi satu fungsi kecil ini menjadi gerbang bagi dua fitur lain.

Kutipan file data.py: fungsi generate_id()

```
def generate_id():
    while True:
        recipe_id = random.randint(100, 999)

        if get_recipe(recipe_id) is None:
            return recipe_id
```

List dan dictionary bekerja bersama di seluruh aplikasi lewat pola yang sama: cook_book sebagai list sumber kebenaran, setiap resep sebagai dictionary berkelas kunci ID, title, ingredients, dan cooking_method, lalu hash table berupa list 10 slot yang isinya simpul dictionary bertingkat gubahan Ikmal, dan trie berupa dictionary bersarang yang dibangun Dhafi dari judul resep. Semua struktur itu membaca satu data yang sama, dan konsistensinya dijaga rebuild_data() yang menyusun ulang hash table serta trie setiap kali cook_book berubah.

6. Penelusuran Kode dan Pengujian

Skenario yang saya telusuri adalah get_recipe_by_number(14) dari fitur Change Recipe. Penelusurannya ringkas karena memang begitu rancangannya, tetapi setiap barisnya berdampak ke fitur lain, seperti terlihat di Tabel 3. Untuk pembuktian jalur gagal, saya ikut sertakan pemanggilan nomor 0 dan 15 yang sama-sama berakhir pada None.

Langkah	Kode yang Dijalankan	Keadaan / Hasil
1	recipe_number = int(input(...))	recipe_number = 14
2	if recipe_number < 1 or ...	14 >= 1 dan 14 <= 14, pemeriksaan batas lolos
3	return cook_book[recipe_number - 1]	mengembalikan cook_book[13]
4	dictionary resep dipakai change_recipe()	Resep Chicken soup ID 503 tampil sebagai resep nomor 14

Tabel 3. Penelusuran get_recipe_by_number(14)

Pengujian saya lakukan dua lapis. Lapis pertama langsung di Python tanpa menu, untuk membuktikan fungsi mengembalikan nilai yang tepat. Lapis kedua lewat program utama, membuktikan nilai itu berguna bagi fitur teman saya. Hasil pengujian langsungnya saya rangkum di Tabel 4, lalu dua skenario menu menyusul setelahnya.

Pengujian langsung di Python	Hasil
get_recipe_by_number(1)	{'ID': 347, 'title': 'Boiled egg', 'ingredients': ['egg', 'water'], 'cooking_method': 'Boiled'}
get_recipe_by_number(14)	{'ID': 503, 'title': 'Chicken soup', 'ingredients': ['chicken', 'curry', 'coconut milk'], 'cooking_method': 'Simmered'}
get_recipe_by_number(0)	None
get_recipe_by_number(15)	None
generate_id() ke-1	226
generate_id() ke-2	580
generate_id() ke-3	594

Tabel 4. Pengujian unit fungsi data resep

Tiga ID acak pada baris terakhir tabel memang berbeda-beda dan tidak pernah sama dengan 14 ID lama, bukti bahwa pemeriksaan lewat get_recipe() bekerja. Berikut dua skenario lewat menu utama yang memakai fungsi-fungsi saya.

Skenario uji 1: menghapus resep nomor 14, nomor divalidasi oleh get_recipe_by_number() (dipadatkan)

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 2

===== REMOVE RECIPE =====
Enter recipe number: 14

     ... resep nomor 1 sampai 13 tampil dengan format yang sama ...
```

```
-------------------------
Recipe Number: 12
Title: Omelette
Ingredients: ['egg', 'onion', 'salt', 'pepper']
Cooking method: Fried
-------------------------

-------------------------
Recipe Number: 13
Title: Baked fish
Ingredients: ['fish', 'lemon', 'salt']
Cooking method: Baked
-------------------------

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 2: menghapus dengan nomor 0 ditolak karena di luar batas

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 2

===== REMOVE RECIPE =====
Enter recipe number: 0
Recipe not found.

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 3: resep baru mendapat ID acak dari generate_id(), terlihat sebagai nomor 15 (dipadatkan)

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 1

===== ADD RECIPE =====
Enter recipe name: Fried banana
```

```
Enter ingredients separated by comma: banana, flour, oil, sugar
Enter cooking method: Fried
Recipe added successfully.

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 5

===== ALL RECIPES =====

-------------------------
Recipe Number: 1
Title: Boiled egg
Ingredients: ['egg', 'water']
Cooking method: Boiled
-------------------------

-------------------------
Recipe Number: 2
Title: Fried egg
Ingredients: ['egg', 'oil']
Cooking method: Fried
-------------------------

     ... resep nomor 3 sampai 14 tampil dengan format yang sama ...

Recipe Number: 14
Title: Chicken soup
Ingredients: ['chicken', 'curry', 'coconut milk']
Cooking method: Simmered
-------------------------
-------------------------
Recipe Number: 15
Title: Fried banana
Ingredients: ['banana', ' flour', ' oil', ' sugar']
Cooking method: Fried
-------------------------

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

7. Modifikasi dan Refleksi

Perbaikan kecil yang saya lakukan adalah pemeriksaan batas pada get_recipe_by_number(). Versi pertamanya langsung mengembalikan cook_book[recipe_number - 1] tanpa diperiksa. Saat diuji dengan nomor 15 program langsung berhenti karena indeksnya keluar dari list, dan yang lebih senyap lagi, nomor 0 ternyata membuat list membaca dari belakang karena indeks -1 di Python memilih elemen terakhir, jadi resep yang salah malah terhapus. Setelah dua syarat batas ditambahkan, kedua kasus itu rapi berakhir pada None dan fitur pemanggilnya mencetak pesan gagal.

Yang saya pelajari dari kejadian itu sederhana tetapi mahal harganya: masukan pengguna tidak pernah bisa dipercaya apa adanya. Bug indeks negatif juga mengajari saya bahwa di Python kesalahan tidak selalu datang sebagai error yang keras, kadang hanya menyaru sebagai program yang tampak benar. Sejak itu saya membiasakan menguji fungsi dengan nilai paling pinggir, bukan hanya nilai yang sopan.

Sepanjang proyek saya memakai AI untuk membantu memahami konsep hash table saat belajar dari materi dan untuk mengecek kemungkinan bug pada fungsi saya. Seluruh kode data.py bagian saya tetap saya tulis sendiri, hasil uji di atas saya jalankan dan saya baca sendiri, dan kalau ditanya fungsi mana pun di bagian saya, saya bisa menjelaskan baris demi barisnya.